

Metodología Detallada de Entrenamiento PPO

Prevención de Data Leaking y Flujo de Datos

Tesis de Portafolios DRL

21 de mayo de 2026

1. Introducción

Este documento detalla exhaustivamente el flujo de datos y la metodología de entrenamiento de los agentes PPO implementados en los scripts `train_ppo.py`, `run_sensitivity.py` y `run_risk_cost_sensitivity.py`. El objetivo principal es auditar el proceso algorítmico para certificar la ausencia de fugas de información (*data leaking*) y explicar paso a paso cómo la red neuronal ingiere los datos, actualiza sus pesos y selecciona el modelo óptimo.

2. Estructura de Datos y Prevención de Data Leaking

La preocupación sobre el *data leaking* (que el modelo vea el futuro) es fundamental en finanzas cuantitativas. En este proyecto, la integridad temporal se garantiza mediante dos barreras estrictas:

2.1. 1. Particionamiento Estricto (DataHandler)

Los datos históricos (`source_data.csv`) se dividen cronológicamente en tres conjuntos estancos:

- **Train (Entrenamiento):** Semanas 0 a 114.
- **Validación (Hyperparameter Tuning):** Semanas 115 a 139.
- **Test (Out-of-Sample):** Semanas 140 en adelante.

Durante la ejecución de la función `agent.train()`, el algoritmo PPO interactúa **única-mente** con el conjunto de Train. El conjunto de Validación se utiliza exclusivamente de forma pasiva a través de un *Callback* para guardar el mejor modelo, y el conjunto de Test permanece completamente invisible y bloqueado en disco hasta la fase de inferencia final.

2.2. 2. Construcción Causal de Features (HMM)

Todas las variables dinámicas (*rolling variables*, como medias, varianzas y covarianzas mixtas generadas por el HMM) fueron calculadas en la etapa de preprocesamiento utilizando **ventanas móviles retrospectivas**. Es decir, el valor de la característica en

el tiempo t se calculó utilizando estrictamente información de los retornos hasta $t - 1$. Cuando el entorno presenta la observación del paso t al agente, no hay absolutamente ninguna información del futuro ($t + 1$) contenida en esas variables.

3. Mecánica del Entorno: ¿Qué entra al modelo paso a paso?

El entrenamiento no procesa la base de datos completa de una vez. El entorno (**PortfolioEnv**) simula el paso del tiempo semana a semana. En cada paso t , el entorno le entrega a la red neuronal (Actor-Critic) un vector de observación O_t :

$$O_t = \left[\frac{x_t}{x_0}, \quad w_{SPX,t-1}, \quad w_{CMC,t-1}, \quad \frac{t}{T}, \quad \text{Features}_t^{HMM} \right] \quad (1)$$

Detalle de la ingesta (Input):

- **Capital Normalizado:** El saldo actual de la cartera dividido por el saldo inicial.
- **Pesos Anteriores:** La distribución de la cartera decidida en el paso anterior (necesaria para calcular cuánto costará mover el portafolio hoy).
- **Tiempo Fraccional:** En qué porcentaje de la simulación nos encontramos (ayuda al agente a saber si el episodio está por terminar).
- **Features HMM:** Las métricas predictivas del estado actual del mercado (Medias, Varianzas, Covarianzas normalizadas).

Con esta información, el Agente (Actor) escoge una acción continua $a_t \in [-1, 1]^2$, la cual el entorno convierte a pesos w_t mediante una proyección Simplex. Luego, el entorno avanza al tiempo $t + 1$, calcula los retornos reales, descuenta los costos de transacción y devuelve una Recompensa (r_t).

4. Dinámica de Aprendizaje: El Algoritmo PPO

El aprendizaje de PPO ocurre mediante un ciclo continuo de Recolecciónz .Actualización” (en *batches*):

4.1. Fase 1: Recolección de Experiencia (Rollout)

El agente juega en el entorno de Entrenamiento durante un bloque fijo de pasos. En tu configuración oficial, está seteado matemáticamente para que el agente juegue exactamente 10 episodios enteros antes de detenerse (`n_steps = 1140`, dado que un episodio tiene 114 semanas). Al jugar episodios completos, evitamos que el agente sufra truncamientos de aprendizaje a mitad de un escenario, almacenando cada transición: (O_t, a_t, r_t, O_{t+1}) .

4.2. Fase 2: Actualización de la Red Neuronal (Batches)

Una vez recolectadas las 1140 experiencias, PPO pausa la simulación y utiliza estos datos para entrenar las dos redes neuronales (MlpPolicy con 2 capas ocultas de 64 neuronas):

1. Mezcla aleatoriamente las 1140 experiencias para romper la correlación temporal.
2. Divide las experiencias en mini-lotes que representan exactamente 1 episodio de datos (`batch_size = 114`).
3. Pasa cada mini-lote por la red para calcular gradientes.
4. Repite este proceso sobre todo el bloque de 1140 datos varias veces (`n_epochs = 10`).

Este proceso ajusta los pesos de la red para maximizar la probabilidad de elegir acciones que resultaron en altas recompensas ajustadas por riesgo, minimizando el impacto de los costos de transacción.

5. Protocolo de Validación y Guardado (EvalCallback)

Para evitar el *overfitting* (que el modelo memorice la serie de tiempo de entrenamiento), los scripts incorporan un `EvalCallback`.

Cada X pasos de entrenamiento (usualmente cada 2000 o 3000 pasos), PPO pausa el entrenamiento y envía una copia determinista (sin exploración aleatoria) de su red neuronal actual a jugar en el entorno de **Validación** (Semanas 115 a 139).

- El agente toma decisiones basadas en la data de validación sin actualizar sus pesos neurológicos (solo evalúa).
- El callback suma la recompensa total obtenida en Validación.
- Si esta recompensa total es superior a la mejor recompensa histórica, el modelo se guarda y sobrescribe el archivo `best_model.zip`.

Por lo tanto, el modelo final resultante de correr el script **NO es necesariamente el modelo del último paso de entrenamiento (paso 57.000)**, sino una "fotografía" de la red en el exacto momento en que fue más generalizable y capaz frente a los datos inéditos de Validación.

6. Conclusión para la Tesis

La metodología implementada es científicamente robusta. No existe *data leaking* porque: 1. Las features de entrada se calcularon ex-ante sin sesgo de mirar al futuro. 2. La arquitectura paso a paso del MDP obliga al agente a recibir un vector de estado cronológico y tomar una decisión antes de observar el retorno del mercado de ese periodo. 3. El uso del set de Validación para el punto de corte (Early Stopping implícito en `best_model`) aísla por completo el proceso de selección de parámetros de los datos finales de Test, legitimando cualquier conclusión de rendimiento extraída de la fase 6 (Batalla Final).